

NestJS

Arquitectura REST API

Módulos · Controladores · Servicios · Repositorios

Guía visual para estudiantes



¿Qué es NestJS?



Framework Node.js

Construido sobre Express. Rápido, escalable y listo para producción.



Arquitectura modular

Cada funcionalidad vive en su propio módulo: separación de responsabilidades.



TypeScript nativo

Tipado fuerte, decoradores y clases. Código más seguro y mantenible.

Inspirado en Angular · Ideal para APIs REST, GraphQL y microservicios

Estructura de carpetas

```
src/
├── main.ts
├── app.module.ts
├── app.controller.ts
├── app.service.ts
├── products/
│   ├── controllers/
│   ├── services/
│   └── repositories/
└── users/
    ├── controllers/
    ├── services/
    └── gateways/
```

Punto de entrada

main.ts arranca la app; app.module.ts es el módulo raíz

Módulos (products/, users/)

Cada dominio tiene su propio módulo independiente

Controllers/

Reciben las peticiones HTTP (GET, POST, etc.)

Services/

Contienen la lógica de negocio

Repositories/

Acceso a datos (base de datos o memoria)

Gateways/

Comunicación con servicios externos (ej: JSONPlaceholder)

Flujo de una petición HTTP



↩ Respuesta viaja de vuelta: Repository → Service → Controller → Cliente



Dependency Injection:

NestJS inyecta automáticamente el Service en el Controller y el Repository en el Service. No tenés que hacer `new Clase()` manualmente.

Módulo Products — en detalle



products/



Módulo

products.module.ts

Registra todos los providers y controllers del dominio. Es el "pegamento" del módulo.



Controller

products.controller.ts

Define las rutas HTTP. Delega el trabajo al Service. Nunca toca datos directamente.



Service

products.service.ts

Contiene toda la lógica de negocio. Llama al Repository para leer/escribir datos.



Repository

products.repository.ts
in-memory-products.repository.ts

Abstrae el acceso a datos. Implementación en memoria para desarrollo/tests.



product.types.ts — Define interfaces y tipos TypeScript compartidos en el módulo

Módulo Users — en detalle



users/



Módulo

users.module.ts

Registra providers del dominio users. Similar a products pero independiente.



Controller

users.controller.ts

Expone rutas /users. Delega al Service para toda la lógica.



Service

users.service.ts

Lógica de negocio de usuarios. Llama al Gateway para obtener datos externos.



Gateway

users.gateway.ts
jsonplaceholder-users.gat
s

Se conecta a APIs externas como JSONPlaceholder. Abstrae la lógica de datos.



Gateway ≠ Repository: Los Gateways se conectan a APIs externas; los Repositories acceden a la base de datos propia.

Repository vs Gateway



Repository

(Módulo: products)

- Gestiona datos propios de la app
- Se conecta a la base de datos interna
- Interfaz: `products.repository.ts`
- Implementación: `in-memory-products.repository.ts`
- Fácil de cambiar de in-memory a SQL/MongoDB
- Patrón: Repository Pattern

vs



Gateway

(Módulo: users)

- Gestiona datos de servicios EXTERNOS
- Se conecta a APIs de terceros (HTTP)
- Interfaz: `users.gateway.ts`
- Implementación: `jsonplaceholder-users.gateway.ts`
- Fácil de cambiar de proveedor externo
- Patrón: Gateway / Adapter Pattern



Ambos patrones permiten cambiar la implementación sin tocar el resto del código — principio de Inversión de Dependencias (SOLID)

Diagrama general de arquitectura



Principios aplicados en esta arquitectura



S

Single Responsibility

Cada clase tiene UNA sola responsabilidad. Controller recibe peticiones, Service ejecuta lógica, Repository accede a datos.



O

Open/Closed

Podés agregar una nueva implementación del Repository sin modificar el Service. La interfaz actúa como contrato.



D

Dependency Inversion

El Service depende de la INTERFAZ (ProductsRepository), no de la implementación concreta. NestJS inyecta la implementación.



M

Modularidad

Cada dominio (products, users) es completamente independiente. Podés agregar, quitar o modificar uno sin afectar el otro.

Resumen — puntos clave



Cada dominio tiene su propio módulo (products, users) con todo lo necesario adentro.



Los Controllers sólo manejan HTTP. Toda la lógica va en el Service.



Los Services son el corazón: contienen las reglas de negocio.



Repository = acceso a base de datos propia. Gateway = acceso a APIs externas.



NestJS inyecta las dependencias automáticamente (Dependency Injection).



Gracias a las interfaces, podés cambiar la implementación sin tocar el resto del código.